

Swinburne University of Technology*School of Science, Computing and Emerging Technologies***MIDTERM COVER SHEET**

Subject Code: COS30008
Subject Title: Data Structures and Patterns
Assignment number and title: Midterm Project: Solution Design & Iterators
Due date: April 16, 2025, 18:59
Lecturer: Dr. Markus Lumpe

Your name: Chrishelle Anthoneyz **Your student ID:** 104067504

Marker's comments:

Problem	Marks	Obtained
1	64	
3	196	
Total	260	

```

1  #include "AutoKey.h"
2  #include <cctype>    // for isalpha and toupper
3
4  // Constructor to constructs the auto key by filtering the
   input keyword
5  AutoKey::AutoKey(const std::string &aKeyword) noexcept
6      : fValue(), fKeyLength(0), fIndex(0)
7  {
8      // Process each character in the provided keyword
9      for (char ch : aKeyword) {
10         if (std::isalpha(static_cast<unsigned char>(ch
11         ))) {
12             fValue.push_back(static_cast<char>(std:::
13             toupper(static_cast<unsigned char>(ch))));
14         }
15     }
16     // Set the original keyword length for use in reset
17     fKeyLength = fValue.size();
18 }
19 // Returns current size of the auto key string
20 size_t AutoKey::size() const noexcept {
21     return fValue.size();
22 }
23 // Returns the character at the current index of the auto
   key string
24 char AutoKey::operator*() const noexcept {
25     return fValue[fIndex];
26 }
27
28 // Increments the internal index to move to the next
   character
29 AutoKey& AutoKey::operator++() noexcept {
30     ++fIndex;
31     return *this;
32 }
33
34 // Returns the current state and then advances the auto
   key iterator
35 AutoKey AutoKey::operator++(int) noexcept {
36     AutoKey temp = *this;
37     ++(*this);
38     return temp;

```

```
39 }
40
41 // Checks if the provided character is an alphabet letter
42 AutoKey& AutoKey::operator+=(char aChar) noexcept {
43     // converts the letter to uppercase and appends it to
the auto key string
44     if (std::isalpha(static_cast<unsigned char>(aChar))) {
45         fValue.push_back(static_cast<char>(std::toupper(
static_cast<unsigned char>(aChar))));
46     }
47     return *this;
48 }
49
50 // Restores the auto key to its initial state by resizing
the key to its original length and resetting the iterator
index to zero
51 void AutoKey::reset() noexcept {
52     fValue.resize(fKeyLength);
53     fIndex = 0;
54 }
55
```

```

1  #include "Vigenereliterator.h"
2  #include <cctype>    // for isalpha, toupper, tolower
3
4  // Constructor to set up the iterator using a member
   initializer list and then prepares the mapping table
5  // if a source text exists, it processes the first
   character appropriately (encode or decode based on the
   mode)
6  Vigenereliterator::Vigenereliterator(const std::string&
   aKeyword,
7
   const std::string&
8
   EVigeneremode aMode)
   noexcept
9      : fMode(aMode),
10      fKeys(aKeyword),
11      fSource(aSource),
12      fIndex(0),
13      fCurrentChar('\0')
14 {
15     // Set up the cipher mapping using the evaluator's
   definition
16     initializeTable();
17
18     // Process the first character from fSource if
   available
19     if (!fSource.empty()) {
20         if (fMode == EVigeneremode::Encode)
21             encodeCurrentChar();
22         else
23             decodeCurrentChar();
24     }
25 }
26
27 // This function enciphers the character at the current
   position in fSource and only processes alphabetic
   characters
28 // The non alphabetic characters are passed unchanged
29 // The function first determines whether the letter was
   lowercase and then converts it to uppercase for processing
   .
30 // It uses the current auto key character to lookup the
   encoded result from fMappingTable,
31 // restores the letter's case if necessary, and stores the

```

```

31  result in fCurrentChar.
32  // Finally, it advances the auto key iterator and then
    appends the processed letter to the key.
33  void VigenereIterator::encodeCurrentChar() noexcept {
34      if (fIndex >= fSource.size()) {
35          fCurrentChar = '\0';
36          return;
37      }
38
39      char current = fSource[fIndex];
40
41      // Non alphabetic characters are not processed by the
    cipher
42      if (!std::isalpha(static_cast<unsigned char>(current
    ))) {
43          fCurrentChar = current;
44          return;
45      }
46
47      // Save whether the current character is lowercase for
    later restoration
48      bool isLower = std::islower(static_cast<unsigned char
    >(current));
49      // Convert the letter to uppercase for cipher
    processing
50      char plainChar = static_cast<char>(std::toupper(
    static_cast<unsigned char>(current)));
51
52      // Retrieve the current letter from the auto key
53      char keyChar = *fKeys;
54      int row = keyChar - 'A';
55      int col = plainChar - 'A';
56
57      // Find the ciphered character using the mapping table
58      char encoded = fMappingTable[row][col];
59
60      // If the original was lowercase, adjust the result
    accordingly
61      if (isLower)
62          encoded = static_cast<char>(std::tolower(
    static_cast<unsigned char>(encoded)));
63      fCurrentChar = encoded;
64
65      // Advance the auto key iterator and append the

```

```

65 uppercase letter to it
66     ++fKeys;
67     fKeys += plainChar;
68 }
69
70 // This function deciphers the character at the current
position in fSource and only processes alphabetic
characters.
71 // Non alphabetic characters are passed unchanged.
72 // Alphabetic characters are converted to uppercase and
decoded using the auto key row in fMappingTable,
73 // Then they are restored to their original case.
74 // Finally the auto key is advanced and updated with the
decoded letter.
75 void VigenereIterator::decodeCurrentChar() noexcept {
76     if (fIndex >= fSource.size()) {
77         fCurrentChar = '\0';
78         return;
79     }
80
81     // Get the current character from fSource
82     char current = fSource[fIndex];
83
84     // Pass through non alphabetic characters
85     if (!std::isalpha(static_cast<unsigned char>(current
))) {
86         fCurrentChar = current;
87         return;
88     }
89
90     // Preserve the original case of the character
91     bool isLower = std::islower(static_cast<unsigned char
>(current));
92
93     // Convert for processing
94     char encodedChar = static_cast<char>(std::toupper(
static_cast<unsigned char>(current)));
95
96     // Get the current key character
97     char keyChar = *fKeys;
98     int row = keyChar - 'A';
99
100    // Perform a linear search through the row to find
the matching character

```

```

101     int col = 0;
102     for (; col < CHARACTERS; col++) {
103         if (fMappingTable[row][col] == encodedChar)
104             break;
105     }
106     char decoded = static_cast<char>('A' + col);
107     if (isLower)
108         decoded = static_cast<char>(std::tolower(
109             static_cast<unsigned char>(decoded)));
110     fCurrentChar = decoded;
111     // Advance and update the auto key with the
112     deciphered letter
113     ++fKeys;
114     fKeys += static_cast<char>(std::toupper(decoded));
115 }
116 // Returns the current ciphered or deciphered character
117 char VigenereIterator::operator*() const noexcept {
118     return fCurrentChar;
119 }
120
121 // Moves the iterator forward by incrementing fIndex,
122 // then processes the next character using either
123 encoding or decoding
124 VigenereIterator& VigenereIterator::operator++() noexcept
125 {
126     fIndex++; // Move to the next character in fSource
127     if (fIndex < fSource.size()) {
128         if (fMode == EVigenereMode::Encode)
129             encodeCurrentChar();
130         else
131             decodeCurrentChar();
132     } else {
133         // Signal the end
134         fCurrentChar = '\0';
135     }
136     return *this;
137 }
138 // Returns a copy of the iterator before incrementing,
139 then advances the iterator
140 VigenereIterator VigenereIterator::operator++(int)
141     noexcept {

```

```

139     VigenereIterator temp = *this;
140     ++(*this);
141     return temp;
142 }
143
144 // Two iterators are considered equal if they point to
the same position in the same source
145 bool VigenereIterator::operator==(const VigenereIterator
    & aOther) const noexcept {
146     return (fIndex == aOther.fIndex &&
147         fSource == aOther.fSource);
148 }
149
150 // Returns an iterator reset to the start of the source
151 VigenereIterator VigenereIterator::begin() const noexcept
    {
152     VigenereIterator iter = *this;
153     iter.fKeys.reset();
154     iter.fIndex = 0;
155     if (!iter.fSource.empty()) {
156         if (iter.fMode == EVigenereMode::Encode)
157             iter.encodeCurrentChar();
158         else
159             iter.decodeCurrentChar();
160     }
161     return iter;
162 }
163
164 // Returns an iterator positioned immediately after the
last character in the source
165 VigenereIterator VigenereIterator::end() const noexcept {
166     VigenereIterator iter = *this;
167     iter.fIndex = fSource.size();
168     iter.fCurrentChar = '\0';
169     return iter;
170 }
171

```